ICSE 2026

# AI Where It Matters

## Where, Why, and How Developers Want AI Support in Daily Work

Rudrajit Choudhuri, Carmen Badea, Christian Bird, Jenna L. Butler, Robert DeLine, Brian Houck

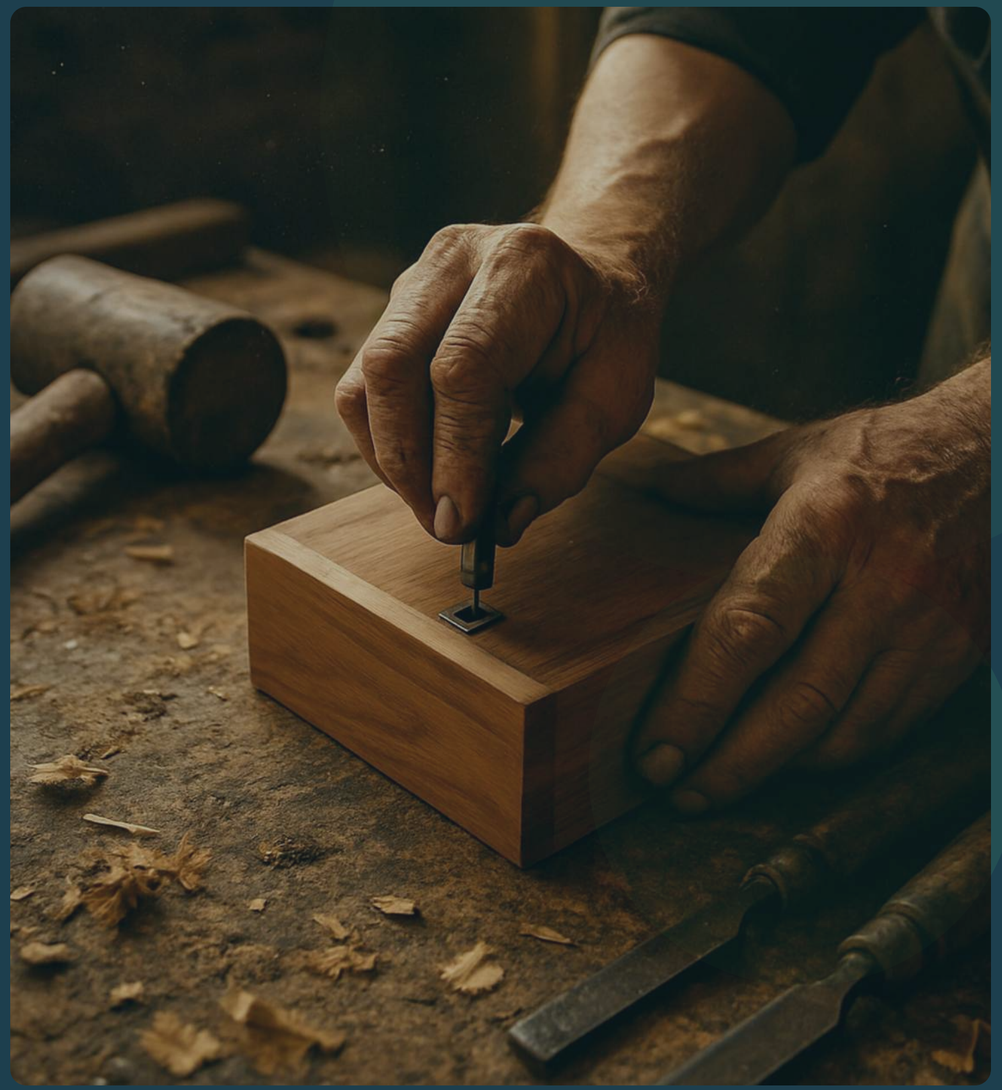
Oregon State University · Microsoft

THE SITUATION

# Developers are using AI more than ever.

# They're spending less time on the work they find meaningful.

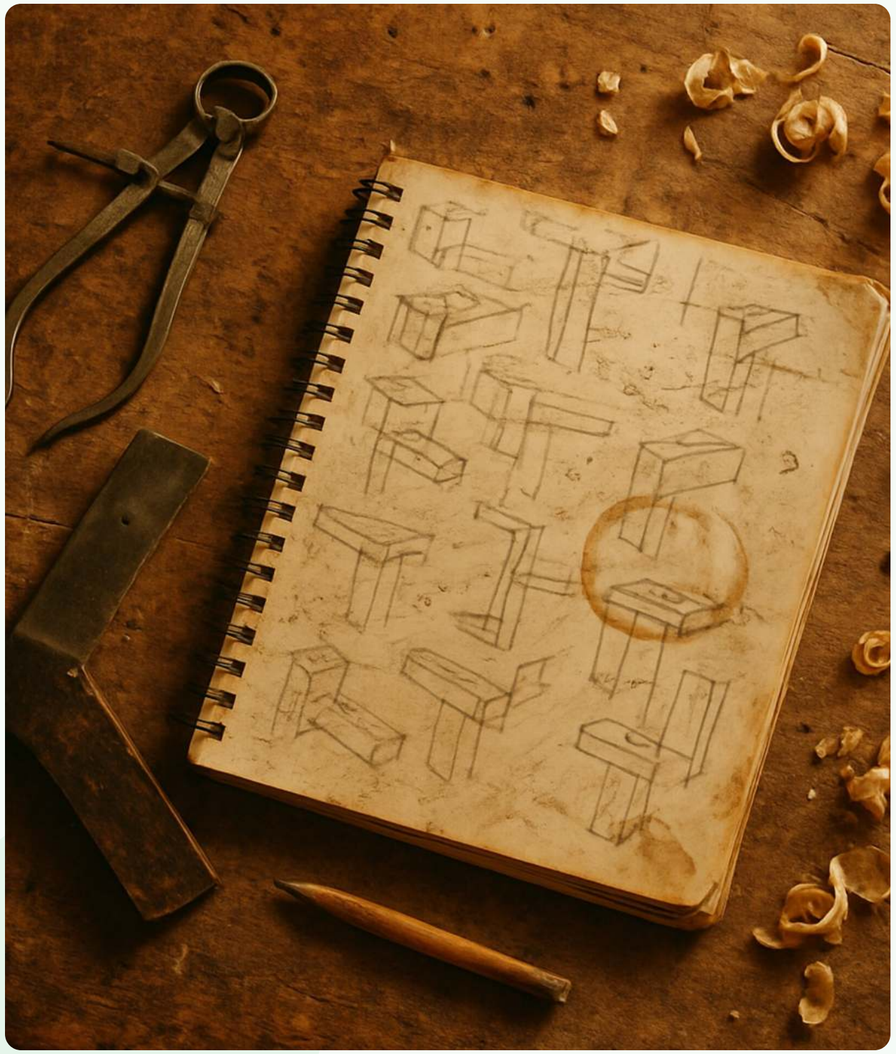
> *"Leave me to do the fun [parts]"*
> — P319, Software Developer



REFRAME

# Not ‘Can AI do this?’ but ‘Should it? Which parts? How much?’

Shifting from capability-fit to a meaning-based account of AI adoption.



METHODOLOGY

# How We Investigated This

A large-scale mixed-methods study.

**Survey**

8,000 developers at Microsoft, July 2025. 1,193 responses (14.86%). 860 valid after exclusions.

**20 Tasks**

Grounded taxonomy across 5 categories: Development, Design & Planning, Quality & Risk, Infrastructure & Ops, Meta-work.

**4 Appraisals**

Each task rated on Value, Identity, Accountability, Demands, plus AI openness and AI usage.

**Analysis**

Mixed-effects regression + reflexive thematic analysis of 1,528 open-ended responses. Member checking with 62 respondents.

METHODOLOGY

# The Survey Instrument: What We Measured

### DEVELOPMENT

Coding | Bug Fixing | Perf Optimization | Refactoring | AI Integration

### DESIGN & PLANNING

System Design | Requirements Engineering | Project Planning

### QUALITY & RISK

Testing & QA | Code Review | Security & Compliance

### INFRASTRUCTURE & OPS

DevOps / CI-CD | Env Setup | Infra Monitoring | Customer Support

### META-WORK

Documentation | Stakeholder Comms | Mentoring | Learning | Research

### FOR EACH TASK, DEVELOPERS RATED:

💎 **Value: Does this matter?**
Higher value → more openness & use. AI complements meaningful tasks.

🪞 **Identity: Is this who I am?**
THE SURPRISE: Less openness but higher actual usage. Guard ownership, use AI to refine craft.

⚖️ **Accountability: Am I on the hook?**
Higher accountability → more openness & use. AI as safeguard under high stakes.

🏋️ **Demands: How hard is this?**
Higher demands → more openness & use. AI as cognitive scaffold.

📊 **Openness to AI**
How open are they to using AI for this task?

📈 **AI Usage**
How much do they use AI for this task today?

EVIDENCE

# Three Worlds of Developer Work

Hierarchical clustering on z-scored appraisal profiles reveals three distinct territories, each with a fundamentally different relationship to AI.

### Core Work

Coding, testing, design, security, code review

**High value, high stakes, high demand. AI as craft amplifier.**

### Ops & Coordination

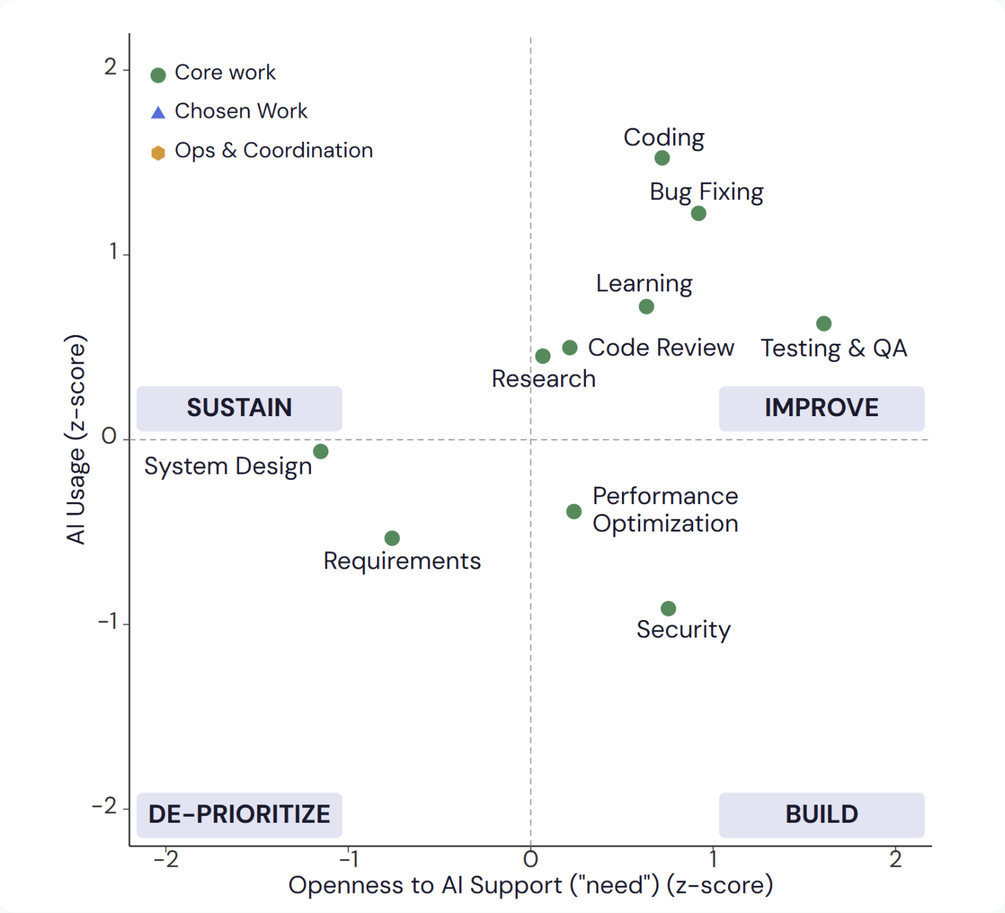
DevOps, env setup, infra, docs, project planning

**Moderate value, weak identity. AI as toil reducer.**

### Chosen Work

Mentoring, AI integration

**Highest identity alignment. Work they find rewarding and want to keep.**



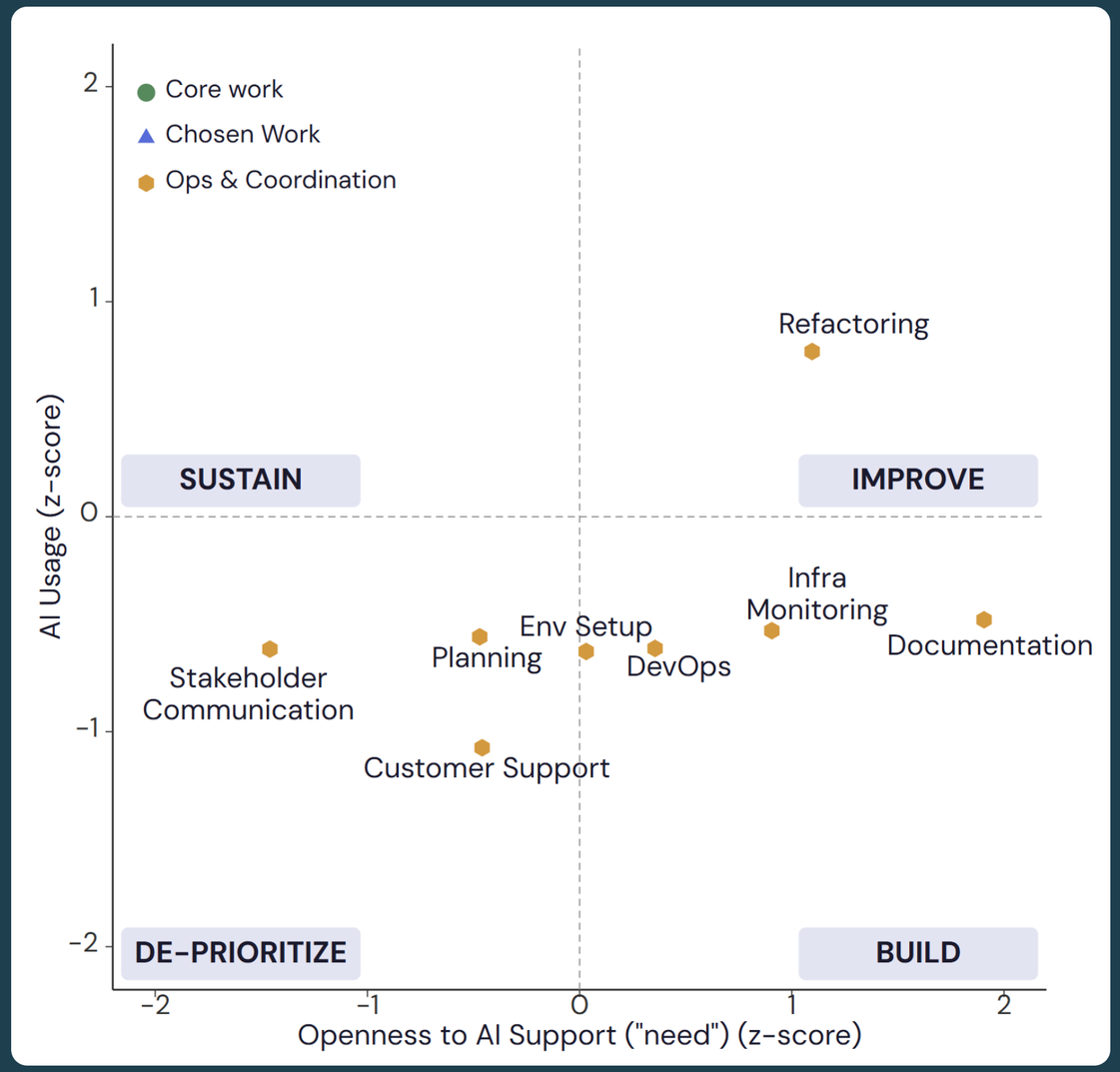

CORE WORK · CLUSTER 1

# Make ME Better

Coding, bug fixing, testing, code review, system design, performance optimization. The highest-value, highest-identity work. Developers want AI to amplify their craft, not replace it.

> *"The focus should be on AI making ME better at my job"*
> — P213

> *"intellectual offloading can result in errors that eventually no one understands"*
> — P409




OPS & COORDINATION · CLUSTER 3

# Keep the Lights On… Carefully

Two kinds of work, two attitudes toward AI. On the right of the quadrant: systems-facing tasks like documentation, monitoring, and DevOps. High openness, underserved by current tools. On the left: human-facing tasks like stakeholder communication and customer support. Much less openness. The dividing line is whether the work faces systems or people.

> *"AI should help in maintenance of services, making sure that the lights are kept on"*
> — P261

> *"Anything that touches prod stays behind human gates"*
> — P165

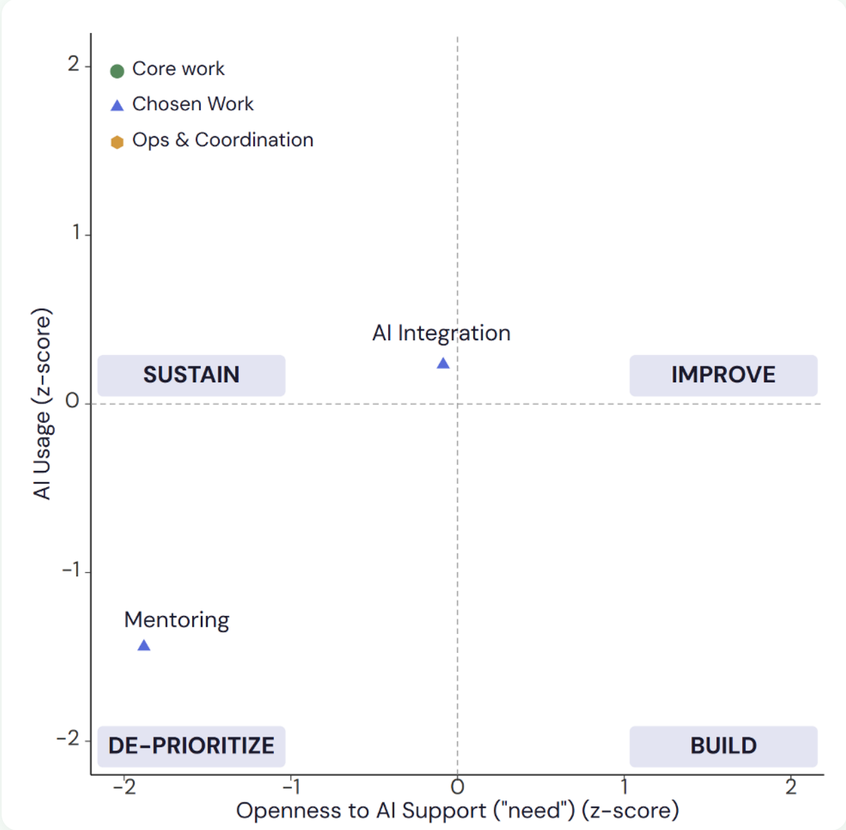
CHOSEN WORK · CLUSTER 2

# I Want to Do This Myself

Mentoring and AI integration. The highest identity alignment of any cluster.

**This is the 10% of the job developers choose to do.**

Mentoring is fulfilling. AI integration is exciting and new. What unifies them is that developers find this work intrinsically rewarding.



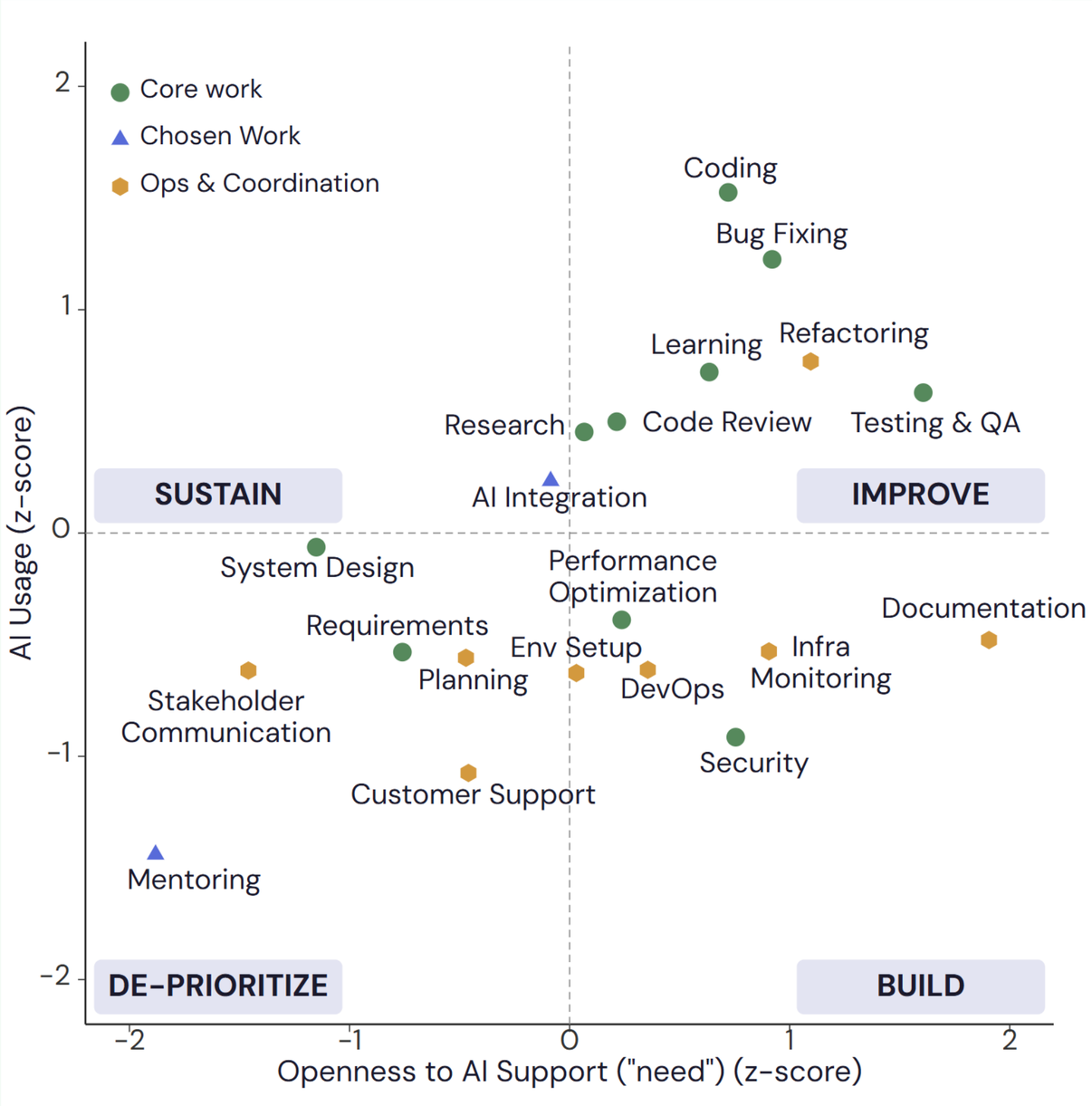

> *"Mentoring teaches the mentor as well... humans need to do it to grow themselves"*
> — P228

> *"I don't want AI to handle AI development, as it brings satisfaction to my work and requires craftsmanship"*
> — P285 (on AI integration)

> *"I don't want it to help mentor people. Relationships are important"*
> — P85

THE MAP

# Where AI Lands

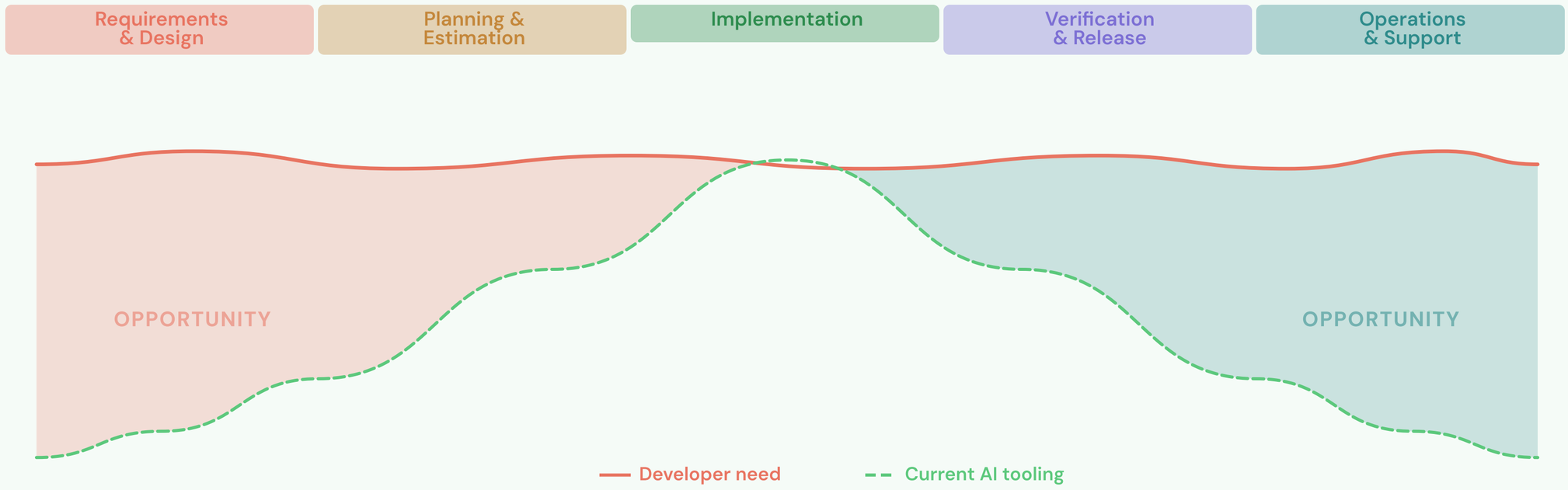



Need vs. Usage across 20 software engineering tasks. N = 860.

THE PATTERN

# The Missing Ends

AI tools cluster in implementation. Developer need spans the full lifecycle. The gap at both ends is where the next systems belong.




*Companion paper: "What's Beyond Copilot? 22 AI Systems Developers Want Built"*

IMPLICATIONS

# What This Means

## FOR RESEARCH

### Appraisal-based framework

Four dimensions (Value, Identity, Accountability, Demands) explain where developers seek and resist AI.

### Identity is the key variable

Less openness but more usage. Developers protect ownership yet use AI to refine craft.

### Meaning endures

AI capabilities shift daily; what endures is what developers find meaningful.

## THE INSTRUMENT

### Run it in your organization

The survey instrument itself is the contribution. Teams can benchmark developer appraisals across 20 tasks, measure their own quadrant, and track how it shifts as tools evolve.

### Toil shifts, not eliminates

Highest returns pair automation with reliability, transparency, alignment.

**Scan to access the materials**

aka.ms/AI-Where-It-Matters

THE PARADOX

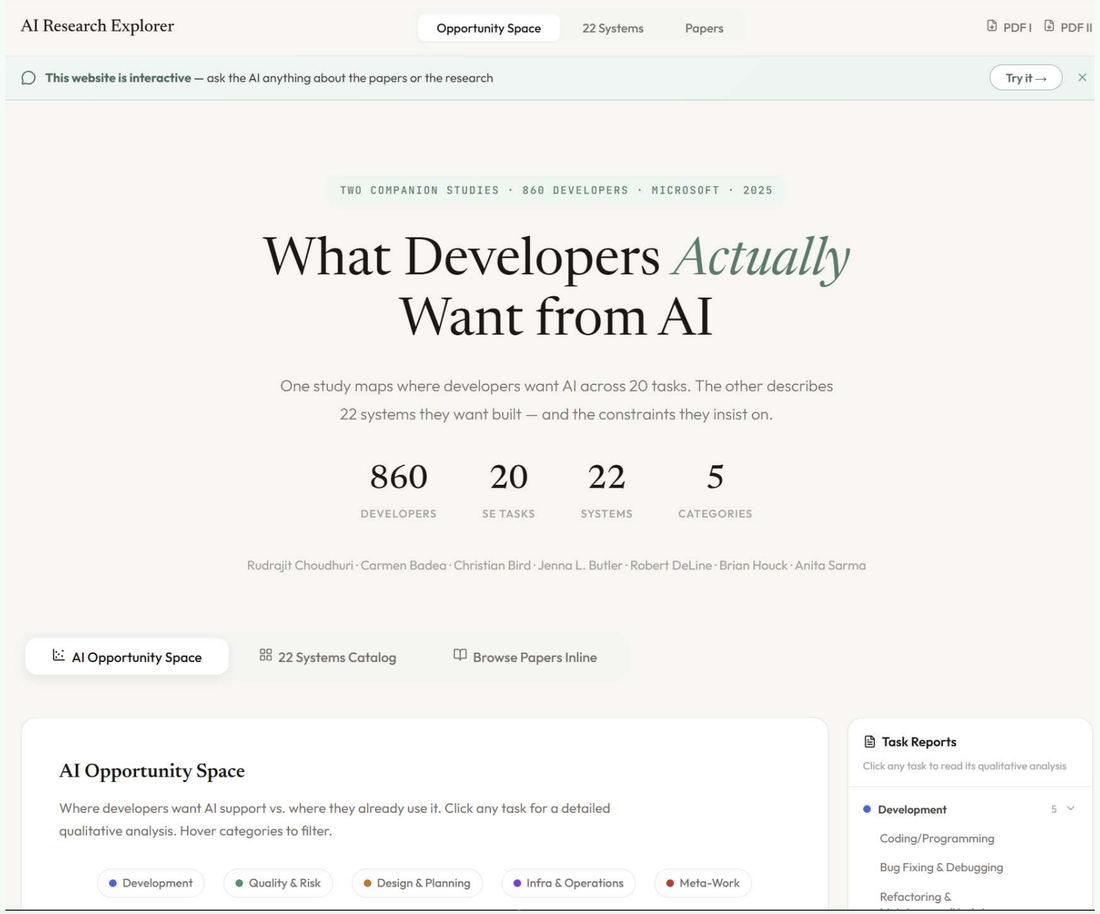
# Developers using AI report higher satisfaction and flow, yet spend less time on the work that defines them.

Designing AI that ignores what work means to developers threatens the very professional identity that makes them effective.

*“Should not turn the worker into a George Jetson.” — P45*

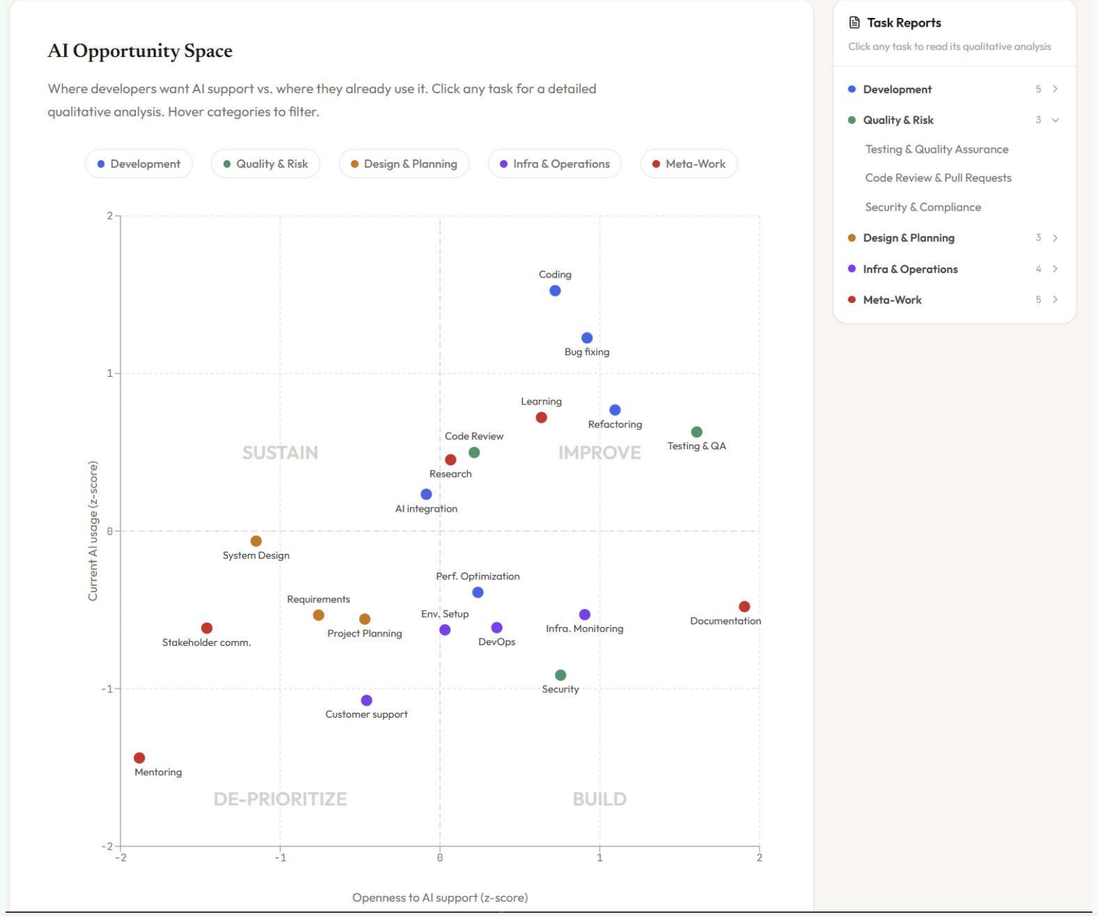
THE WEBSITE

# Explore It Yourself



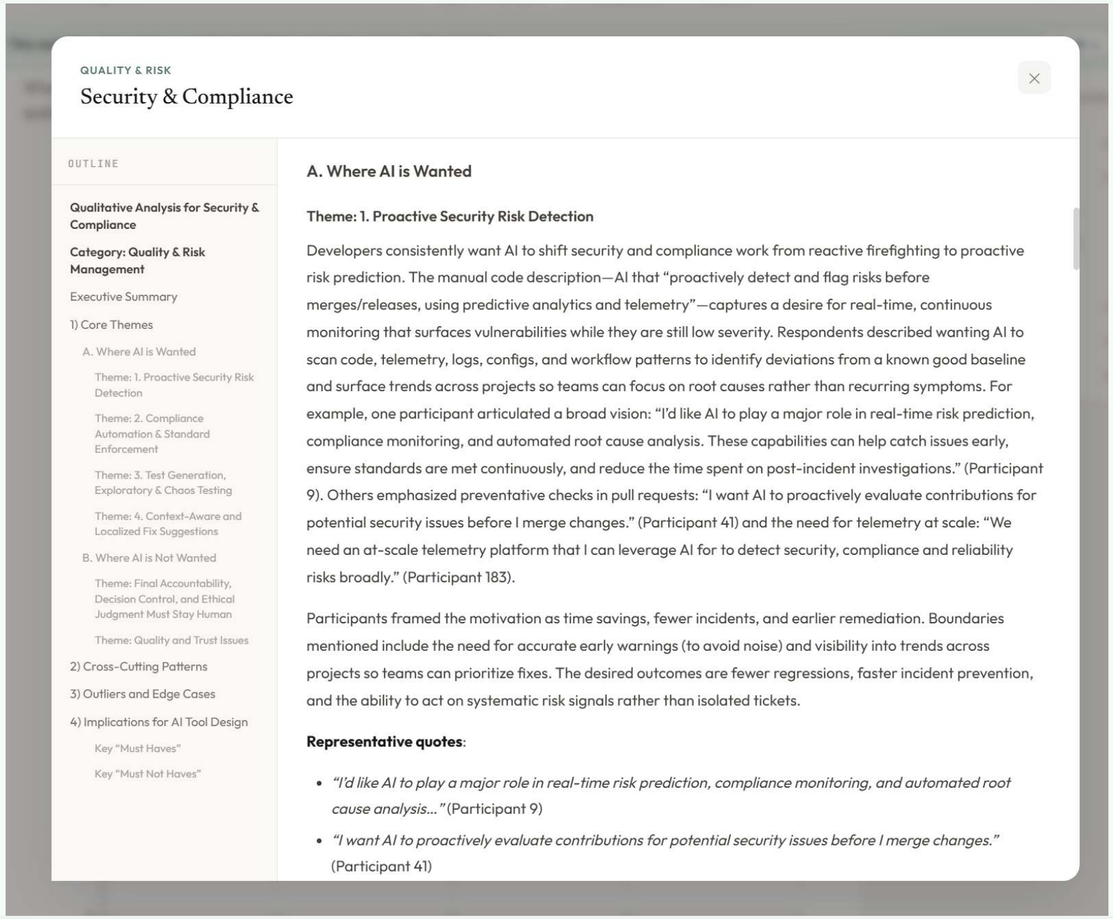
THE WEBSITE

# Interactive Opportunity Space



THE WEBSITE

# Task Deep Dives



QUALITY & RISK

## Security & Compliance

OUTLINE


**Qualitative Analysis for Security & Compliance**

**Category: Quality & Risk Management**

Executive Summary

1) Core Themes

A. Where AI is Wanted

Theme: 1. Proactive Security Risk Detection

Theme: 2. Compliance Automation & Standard Enforcement

Theme: 3. Test Generation, Exploratory & Chaos Testing

Theme: 4. Context-Aware and Localized Fix Suggestions

B. Where AI is Not Wanted

Theme: Final Accountability, Decision Control, and Ethical Judgment Must Stay Human

Theme: Quality and Trust Issues

2) Cross-Cutting Patterns

3) Outliers and Edge Cases

4) Implications for AI Tool Design

Key "Must Haves"

Key "Must Not Haves"


### A. Where AI is Wanted

**Theme: 1. Proactive Security Risk Detection**

Developers consistently want AI to shift security and compliance work from reactive firefighting to proactive risk prediction. The manual code description—AI that "proactively detect and flag risks before merges/releases, using predictive analytics and telemetry"—captures a desire for real-time, continuous monitoring that surfaces vulnerabilities while they are still low severity. Respondents described wanting AI to scan code, telemetry, logs, configs, and workflow patterns to identify deviations from a known good baseline and surface trends across projects so teams can focus on root causes rather than recurring symptoms. For example, one participant articulated a broad vision: "I'd like AI to play a major role in real-time risk prediction, compliance monitoring, and automated root cause analysis. These capabilities can help catch issues early, ensure standards are met continuously, and reduce the time spent on post-incident investigations." (Participant 9). Others emphasized preventative checks in pull requests: "I want AI to proactively evaluate contributions for potential security issues before I merge changes." (Participant 41) and the need for telemetry at scale: "We need an at-scale telemetry platform that I can leverage AI for to detect security, compliance and reliability risks broadly." (Participant 183).

Participants framed the motivation as time savings, fewer incidents, and earlier remediation. Boundaries mentioned include the need for accurate early warnings (to avoid noise) and visibility into trends across projects so teams can prioritize fixes. The desired outcomes are fewer regressions, faster incident prevention, and the ability to act on systematic risk signals rather than isolated tickets.

**Representative quotes**:

- *"I'd like AI to play a major role in real-time risk prediction, compliance monitoring, and automated root cause analysis..."* (Participant 9)
- *"I want AI to proactively evaluate contributions for potential security issues before I merge changes."* (Participant 41)

THE WEBSITE

# 22 Systems Developers Want Built

## 22 Systems Developers Want Built

Sort by prevalence

Click any card for full details, constraints, and developer quotes.

All (22) | Development (4) | Quality & Risk (5) | Design & Planning (5) | Infra & Operations (4) | Meta-Work (4)

DEVELOPMENT 50.1% of 353
### Scoped-PR Builder
Technical debt accumulates in mature codebases because careful maintenance at scale produces no...

META-WORK 45.9% of 157
### DocSync
Documentation drifts unnoticed from the code it describes; engineers rarely update docs when they...

QUALITY & RISK 44.5% of 155
### Change-Aware Test Generator
Test suites lag behind code, leaving coverage gaps as behavior changes outpace test authoring.

INFRA & OPERATIONS 40.6% of 101
### Telemetry Assistant
Alert fatigue and incident orientation overwhelm on-call engineers; the first hour of an incident goes to...

INFRA & OPERATIONS 33.7% of 101
### CI/CD Blueprint Builder
Pipeline definitions are hard to author and harder to debug; engineers waste hours on configuration that...

META-WORK 30.6% of 157
### Ramp-Up Coach
Onboarding engineers spend weeks orienting to a new codebase with documentation that is generic,...

DESIGN & PLANNING 30.5% of 223
### Design-To-Sprint Workbench
Turning an accepted design plan into a structured backlog, dependency map, and sprint sequence is...

DEVELOPMENT 27.8% of 353
### Proactive QA Gate
Quality gates are missing during authorship because there's no real-time error detection or test coverag...

DESIGN & PLANNING 27.8% of 223
### Architecture Studio
Design sessions often produce alternatives without a structured comparison; teams lack a tool that...

QUALITY & RISK 23.2% of 155
### PR Review Assistant
Large PRs are hard to review: intent is unclear, risk boundaries are implicit, and mechanical churn...

QUALITY & RISK 21.9% of 155
### Security Advisor
Current tools flag issues, but vulnerability findings arrive late, lack context and actionable fixes,...

DESIGN & PLANNING 20.6% of 223
### Design Analyzer
Design reviews surface obvious issues but miss hidden dependencies, non-functional requirements gaps,...

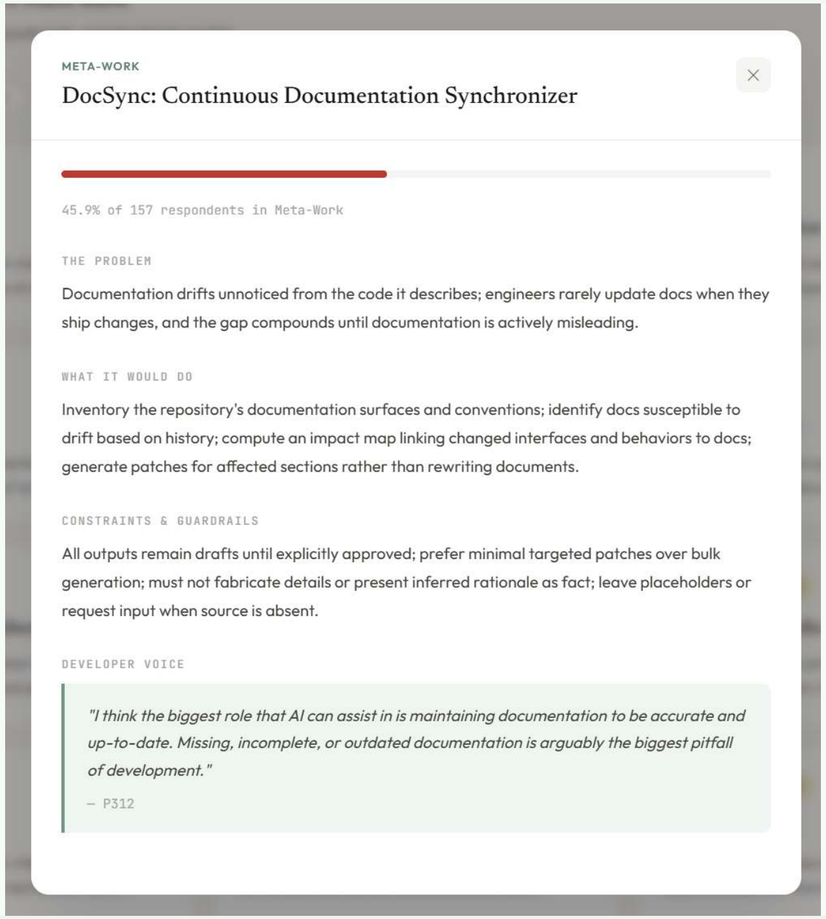
DESIGN & PLANNING 19.7% of 223

DEVELOPMENT 19.5% of 353

DEVELOPMENT 18.4% of 353

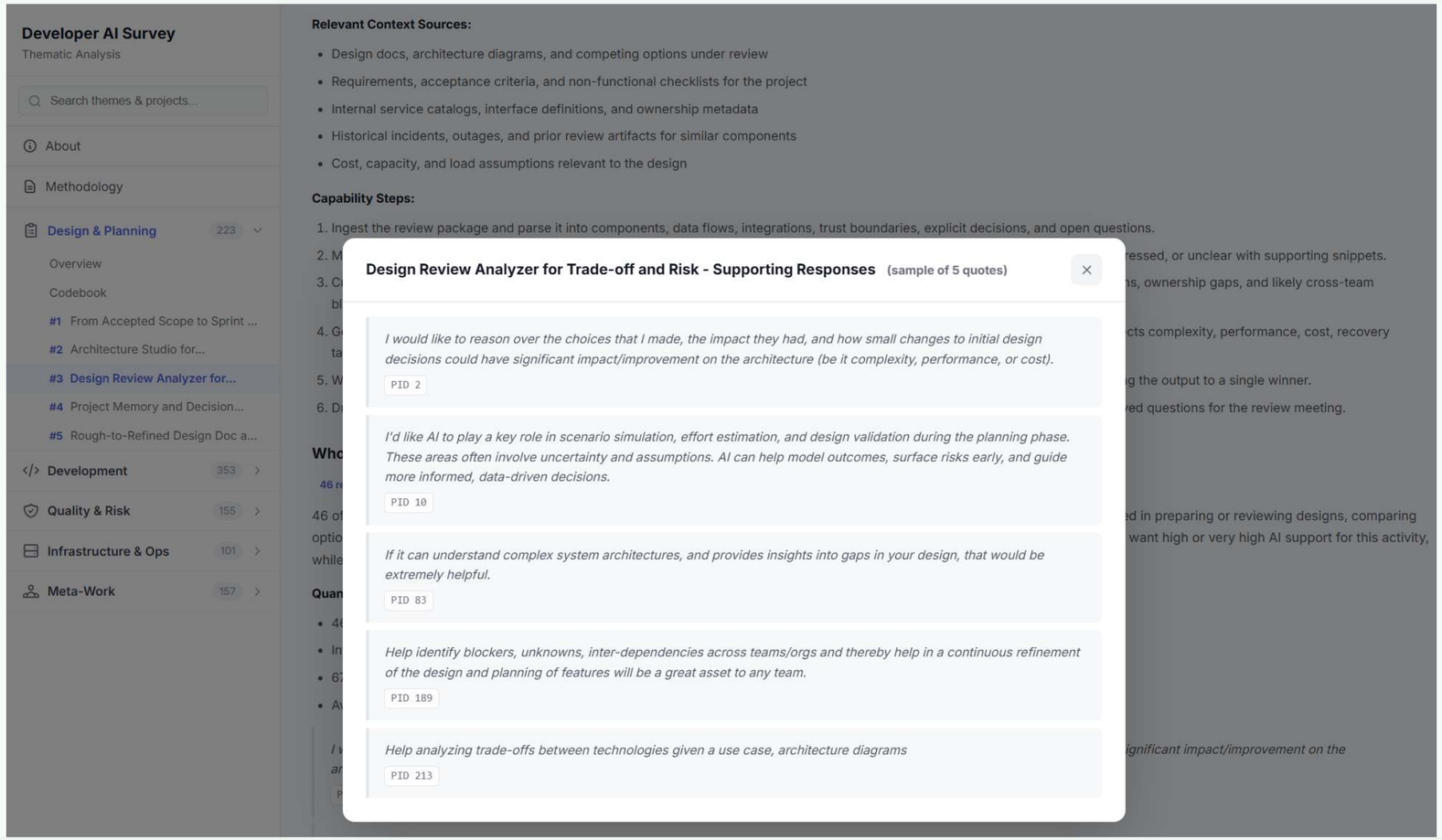
THE WEBSITE

# Every System, Documented



THE WEBSITE

# Grounded in Developer Voices



COMPANION PAPER

# To Copilot and Beyond

## To Copilot and Beyond: 22 AI Systems Developers Want Built

Rudrajit Choudhuri[1] Christian Bird[2] Carmen Badea[2] Anita Sarma[1]
[1]Oregon State University, OR, USA. Email: {choudhru, anita.sarma}@oregonstate.edu
[2]Microsoft Research, WA, USA. Email: {cbird, cabadea}@microsoft.com

### Abstract

Developers spend roughly one-tenth of their workday writing code, yet most AI tooling targets that fraction. This paper asks what should be built for the rest. We surveyed 860 Microsoft developers to understand where they want AI support, and where they want it to stay out. Using a human-in-the-loop, multi-model council-based thematic analysis, we identify 22 AI systems that developers want built across five task categories. For each, we describe the problem it solves, what makes it hard to build, and the constraints developers place on its behavior.

Our findings point to a growing *right-shift burden* in AI-assisted development: developers wanted systems that embed quality signals earlier in their workflow to keep pace with accelerating code generation, while enforcing explicit authority scoping, provenance, uncertainty signaling, and least-privilege access throughout. This tension reveals a pattern we call *"bounded delegation"*: developers wanted AI to absorb the assembly work surrounding their craft, never the craft itself. That boundary tracks where they locate professional identity, suggesting that the value of AI tooling may lie as much in where and how precisely it stops as in what it does.

### 1 Introduction

When asked what software developers do, most people picture someone writing code. AI tooling ventures have largely followed the same intuition, concentrating investment on code generation [14, 54]. But studies of how developers actually spend their time tell a different story: writing code accounts for roughly one-tenth of their day [29, 35]. The rest, spent debugging production incidents, navigating compliance, onboarding teammates, keeping documen-

supposed to deliver. The downstream costs compound: developer fatigue and burnout [20, 36], and a growing accumulation of AI-induced technical debt [18, 34].

Designing tools that address this problem requires knowing what developers actually need, and that requires their voice. Decades of work-design research show that automation-first approaches cap gains at labor arbitrage; augmentation is what expands them [3–5, 12]. When workers' perspectives inform tool design, organizations achieve more sustainable improvements in both productivity and well-being [24, 49].

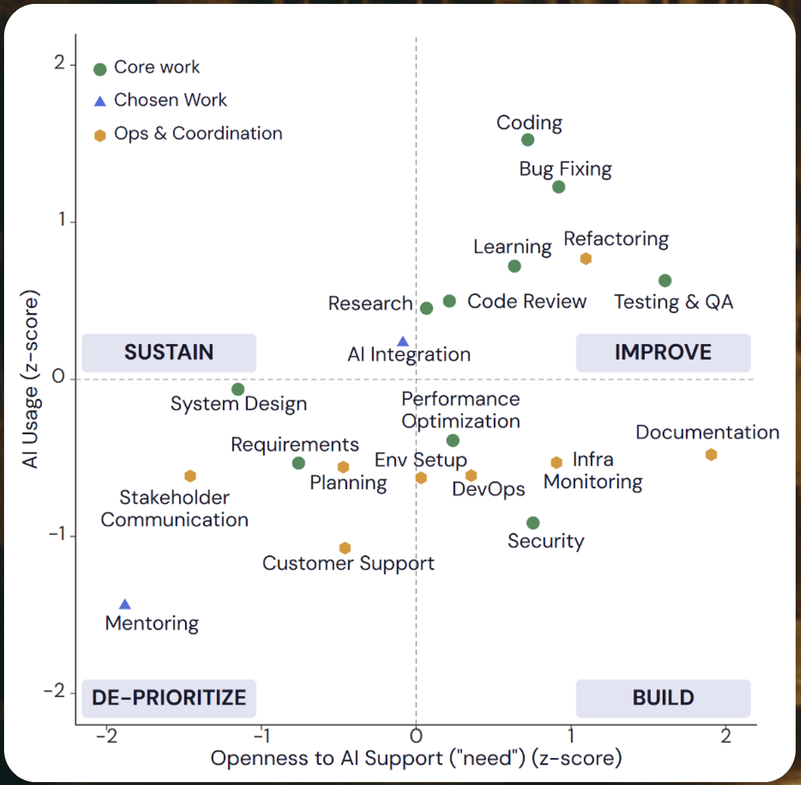
Yet most studies of AI in software engineering examine current adoption patterns, productivity effects, or task-level exposure [2, 13, 17, 20, 28, 29, 31, 44]. They tell us where developers use AI and why they adopt or resist it. They do not tell us what should be built next, what those systems should do, or what they should never do on their own. This paper provides that study. We ask:

**RQ**: *What AI systems do developers want built, and what conditions do they place on those systems for them to be acceptable?*

The data comes from a large-scale survey of 860 Microsoft developers reported in [16], which investigated how developers' cognitive appraisals of their daily work shape their openness to and use of AI. This paper asks the complementary question: what, concretely, should be built? Using a human-in-the-loop, multi-model council-based thematic analysis [48, 52] of the survey's open-ended responses, we identified 22 AI systems across five software development task categories that developers want built. For each, we describe the problem developers want solved, what makes it hard

GO EXPLORE

# AI Where It Matters




aka.ms/AI-Where-It-Matters

📄 **The Paper** · ICSE 2026

📘 **Companion Paper** · 22 AI Systems Developers Want

🔗 **The Website** · explore tasks, chat with the papers

📋 **Task Deep Dives** · per-task qualitative reports

Rudrajit Choudhuri, Carmen Badea, Christian Bird, Jenna L. Butler, Robert DeLine, Brian Houck · ICSE 2026